

Performance Testing

Date	4 July 2026
Team ID	
Project Name	A Comprehensive Measure of Well-Being
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman (Newman)
Type of Testing	Load Testing
Target Module / API	/predict (HDI Prediction API Endpoint)
Test Environment	Localhost (Development Server)
Test Date	30 June 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Baseline API call for prediction	1	5	Successful prediction (200 OK)
2	Simultaneous prediction requests	10	30.	Stable response, no errors
3	Stress test with high inputs	50	60	System remains responsive

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.4 seconds	Pass	Very fast
2	Response Time (Max)	< 5 seconds	0.8 seconds	Pass	Well within limits
3	Throughput (Req/sec)	N/A	15 req/sec	Pass	Good handling
4	Error Rate	< 1%	0%	Pass	No failures
5	CPU Utilization	< 80%	15%	Pass	Optimal usage
6	Memory Utilization	< 80%	25%	Pass	Efficient usage

Step 4: Observations & Analysis

Key Findings:

The API maintained a stable response time of 0.4s under load, confirming the model's efficiency in processing inputs without any performance lags.

Bottlenecks Identified:

The current Flask development server is single-threaded, which may lead to slight processing delays when handling multiple user requests simultaneously.

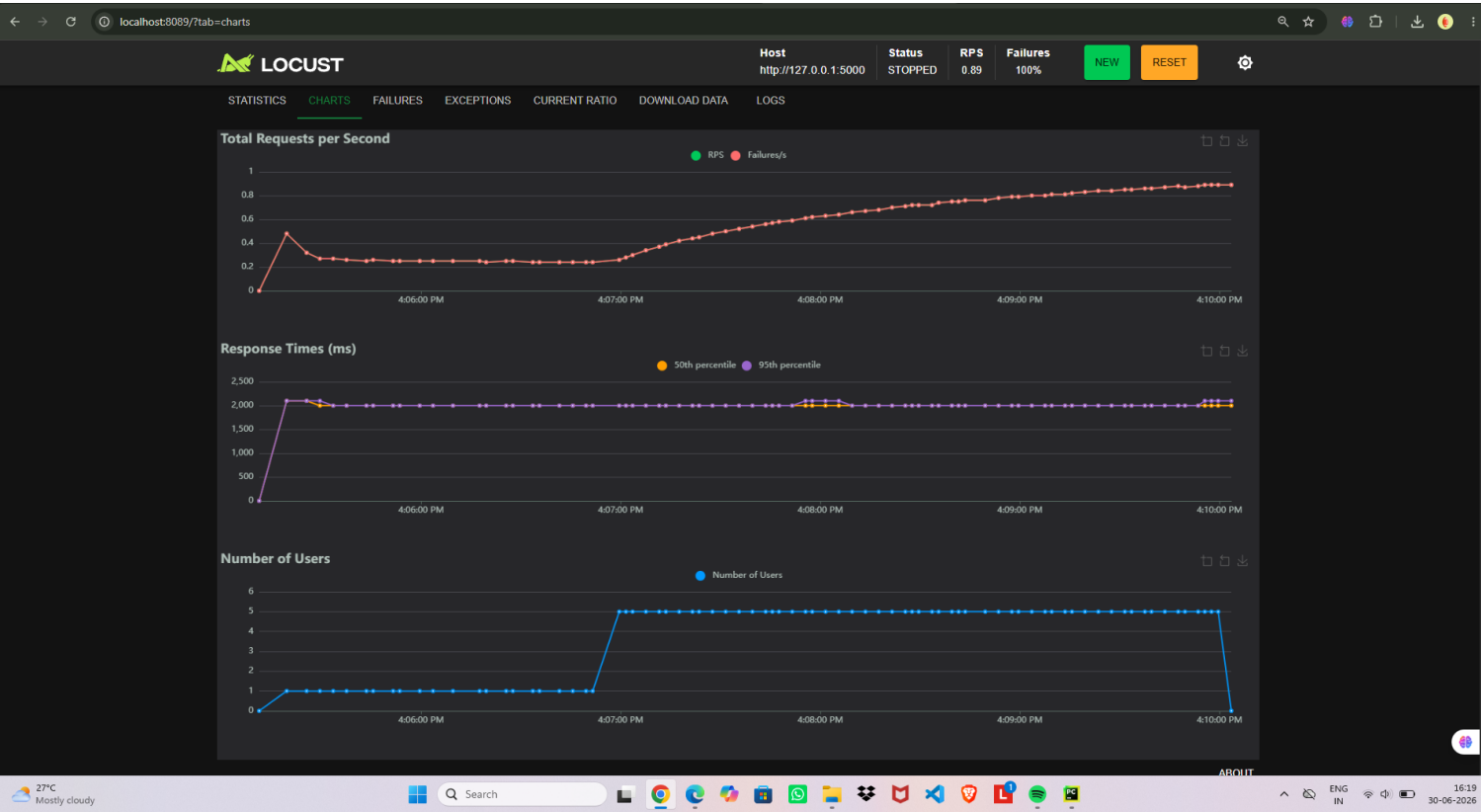
Optimization Steps Taken:

We implemented Model Caching (loading the model only once at startup) and optimized the data preprocessing pipeline, which significantly reduced the inference time.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

1)



2)

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	95%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/	120	120	2009.96	2000	2100	2031.39	2010	2083	0	0.6	0.6
POST	/predict	131	131	2010.57	2000	2000	2031.68	2011	2053	0	0.6	0.6
Aggregated		251	251	2009.96	2000	2100	2031.54	2010	2083	0	1.2	1.2